

Abstract

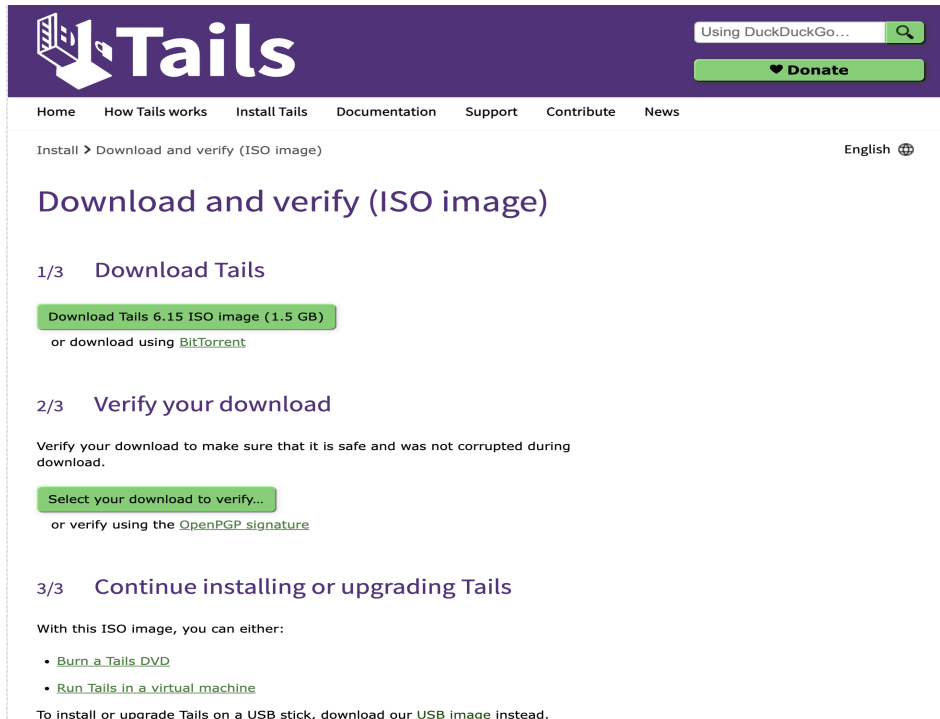
Use the Tails operating system and the Tor network to explore how anonymous communication and privacy protection technologies function in practice. Through hands-on use, detailed packet analysis, and investigation of the tools provided by Tails, we get an understanding of how Tor (The Onion Router) ensures secure, encrypted, and anonymous browsing. This exercise will help understand internet anonymity, traffic routing, and the strengths and limitations of using Tor with a security focused operating system like Tails.

Introduction

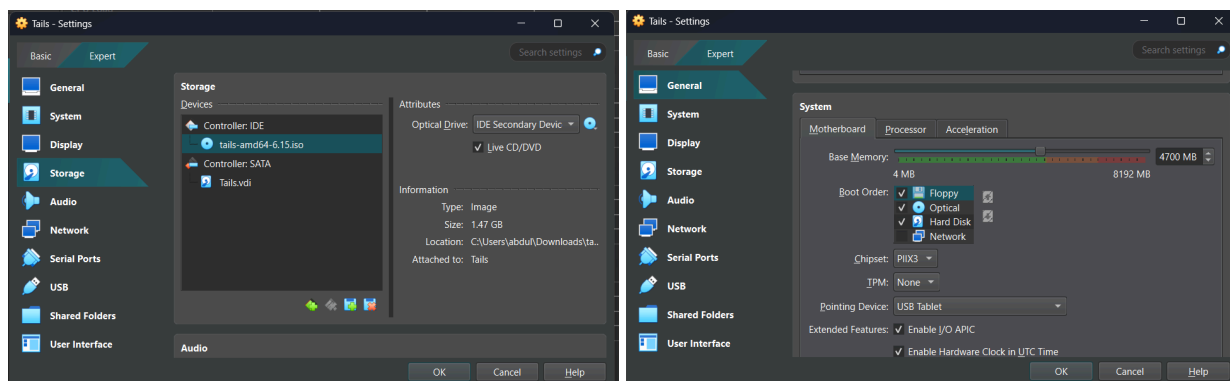
Tails Operating System will be used in a VirtualBox Virtual machine. Wireshark will be used for packet analysis in Kali Linux OS for further analysis. Using Onion Share to securely share files. And tcp-dump, a command-line network packet analyzer to capture packets that pass through a network interface and lets you analyze them. WireShark runs as root while capturing on the 'any' adapter in the background from the kali operating system .

Summary of Results

Starting WireShark in Kali Linux. Selecting the 'any' interface for packet capture. And now downloading tails iso image by going to the Tails webpage using the link provided: <https://tails.net/install/download-iso/index.en.html>. Then after you download the ISO image. Open virtual box and click the option 'Machine' then select the option 'new' after clicking it you will be allowed to select the ISO image file after selecting you will be directed to settings of Virtual machine. When prompted to settings in VirtualBox, make sure to enable the 'Live CD/DVD' option under the system settings. This ensures that the virtual machine boots directly from the ISO file without attempting to install anything to a hard drive, which aligns with Tails purpose of leaving no digital trace. Additionally, allocate enough system memory (at least 2048 MB) to ensure that the Tails OS runs smoothly during our session. Without sufficient memory, performance may lag or certain features may not function correctly. We have allocated about 4700 MB which is like around 4.5GB of RAM. And another concern about insufficient memory is the VM will start using the Hard Disk to run some files which can lead to leave digital traces of our work on the harddisk.



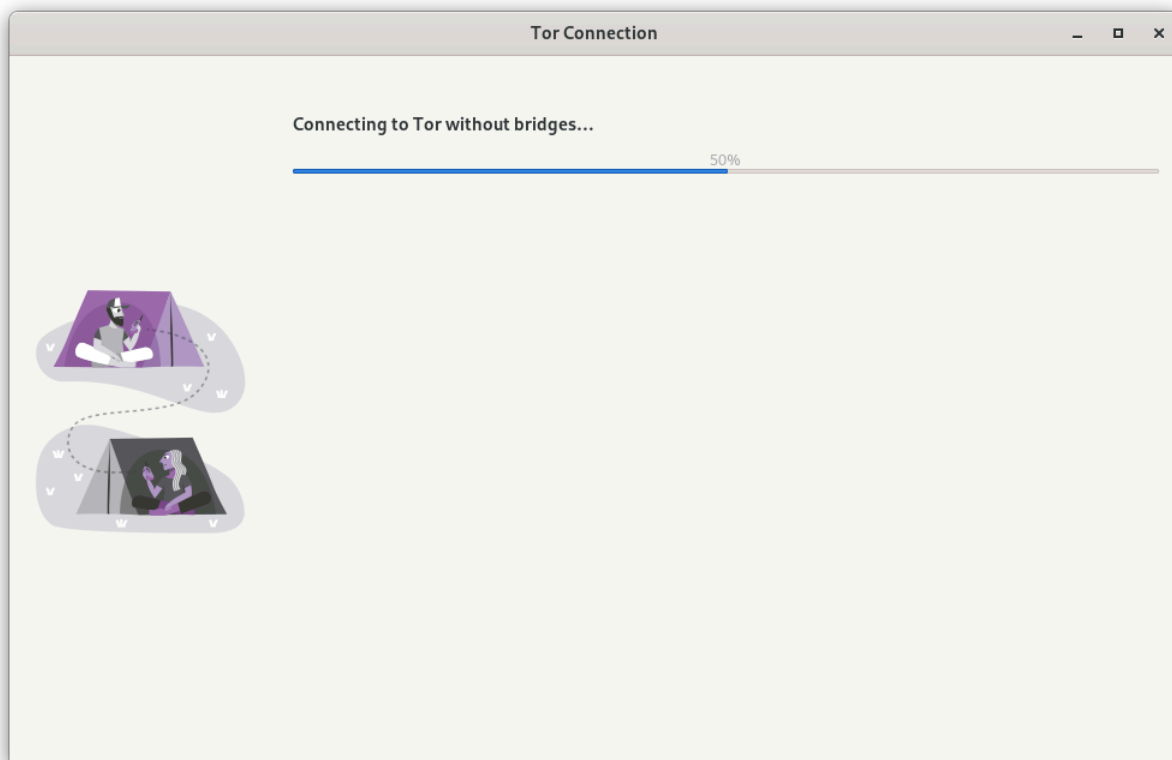
This is the image of Tails website and we can see the option to download the ISO image.



These images show the Tails VM setting in the virtual box, the first image shows that we are selecting the option Live CD/DVD. And in the second we can see that the allocation of memory upto 4700 MB, which would be enough for today's session.

Now after downloading and setting up the settings we are going to start the machine, once started we could see a few options where we have to set the Tails Administration Password, be sure to set a sudo password. This is required for sudo. We have successfully set the administrator password and after that we click start Tails. When starting Tails, the system will prompt the user to choose how they want to connect to the

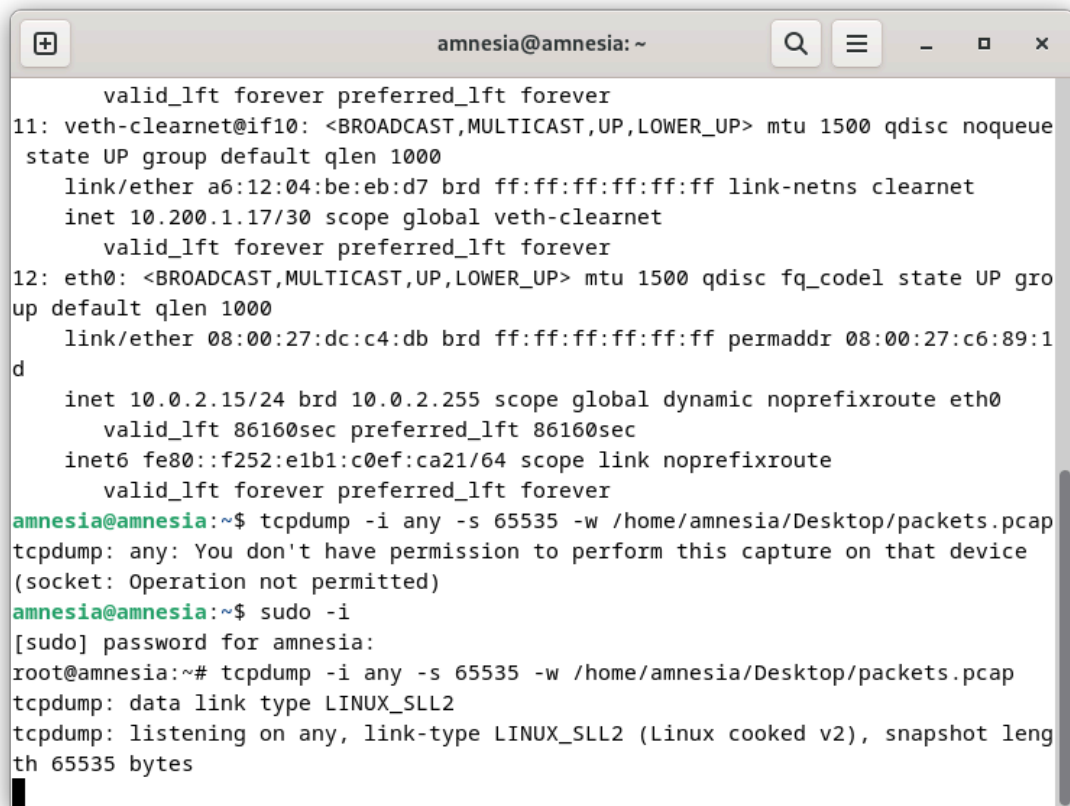
Tor network either directly or through a bridge. The default option is to connect automatically without a bridge, which is suitable for users in countries or networks that do not block access to Tor. This option is faster and simpler but may not work in environments where Tor is censored. Alternatively, users can choose to connect using a bridge, which is a special type of Tor relay that helps bypass network restrictions and hide Tor usage. This is especially useful in countries or institutions that actively block Tor traffic. By using a bridge, the user can avoid detection and censorship, although it may require additional configuration and can be slightly slower. When we connect without a bridge, our ISP can see we are connecting to TOR but couldn't see what we are doing on TOR. We can also connect to VPN if we want to hide our connection to Tor from our ISP.



The above image shows our Tails OS connecting to Tor circuit without Bridges.

After we are connected we are going to open the terminal from the application tab of Tails, the typing `ip addr` command, to see our ip address. We can see that our inet is '10.0.2.15'. Then we are going to start the `tcpdump`, it is a powerful command-line packet analyzer used to capture and inspect network traffic in real time. Before we start `tcpdump` we need root access, so we type '`sudo -i`' this will make us a sudo user. After we start it using the command "`tcpdump -i any -s 65535 -w /home/amnesia/Desktop/packets.pcap`". This term '`-i any`' captures traffic on all available interfaces. '`-s 65535`' sets the snapshot length to capture full packets. '`-w`

/home/amnesia/Desktop/packets.pcap' saves the output to a file in .pcap format, which is compatible with Wireshark for later analysis.

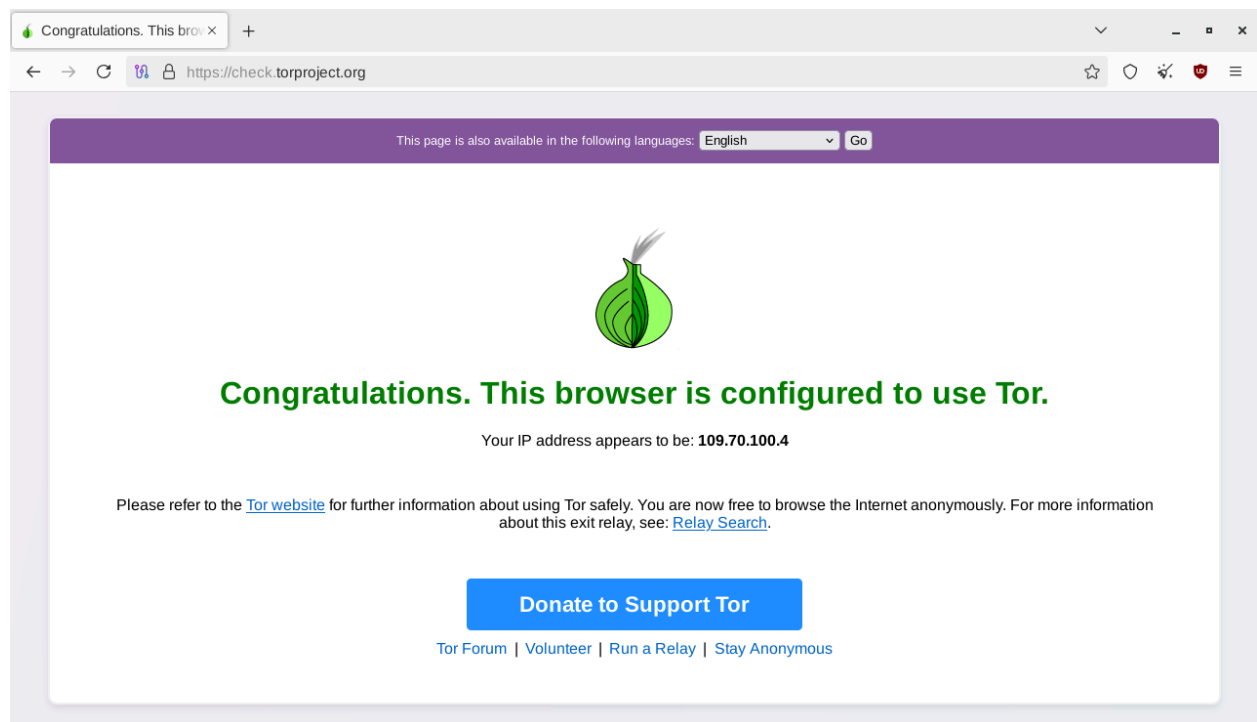
A terminal window titled 'amnesia@amnesia: ~' showing network configuration for veth and eth0 interfaces, followed by an attempt to run tcpdump as a regular user (which fails with a permission error) and then as root using sudo (which succeeds).

```
valid_lft forever preferred_lft forever
11: veth-clearneth@if10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue
state UP group default qlen 1000
    link/ether a6:12:04:be:eb:d7 brd ff:ff:ff:ff:ff:ff link-netns clearnet
    inet 10.200.1.17/30 scope global veth-clearneth
        valid_lft forever preferred_lft forever
12: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 08:00:27:dc:c4:db brd ff:ff:ff:ff:ff:ff permaddr 08:00:27:c6:89:1
d
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 86160sec preferred_lft 86160sec
    inet6 fe80::f252:e1b1:c0ef:ca21/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
amnesia@amnesia:~$ tcpdump -i any -s 65535 -w /home/amnesia/Desktop/packets.pcap
tcpdump: any: You don't have permission to perform this capture on that device
(socket: Operation not permitted)
amnesia@amnesia:~$ sudo -i
[sudo] password for amnesia:
root@amnesia:~# tcpdump -i any -s 65535 -w /home/amnesia/Desktop/packets.pcap
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot leng
th 65535 bytes
```

The above image shows a terminal session on the Tails operating system where we are attempting to run `tcpdump` to capture network packets. Initially, the command `tcpdump -i any -s 65535 -w /home/amnesia/Desktop/packets.pcap` is executed under the regular user `amnesia`, but it fails due to a permission error: "Operation not permitted." This happens because capturing packets on network interfaces requires elevated privileges. To resolve this, we switch to the root account using `sudo -i`, enter the administrative password, and successfully rerun the `tcpdump` command. Now, `tcpdump` begins listening on all interfaces (`-i any`) and writes full packet data (`-s 65535`) to a `.pcap` file saved on the Desktop. The confirmation message indicates that the capture is running using `LINUX\_SLL2` (a cooked capture interface) with a snapshot length of 65535 bytes, meaning the full packets are being recorded. This packet capture can later be analyzed in tools like Wireshark.

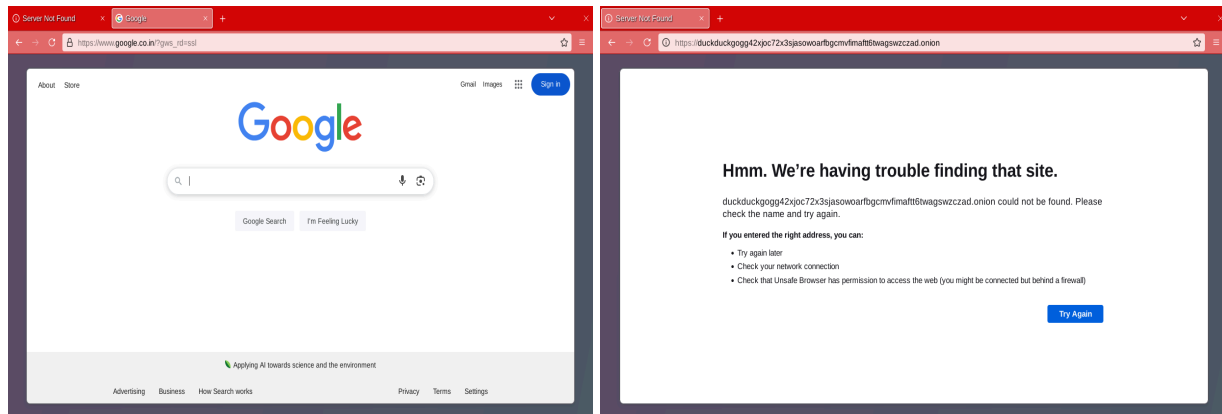
After we start tcpdump we go to the browser and type "<https://check.torproject.org/>" this will help us check our IP address and compare it to known Tor exit nodes. Based on this check, it displays the message: "Congratulations. This browser is configured to use Tor."

This means our browser is successfully routing traffic through the Tor network and your connection is anonymous (to a degree).

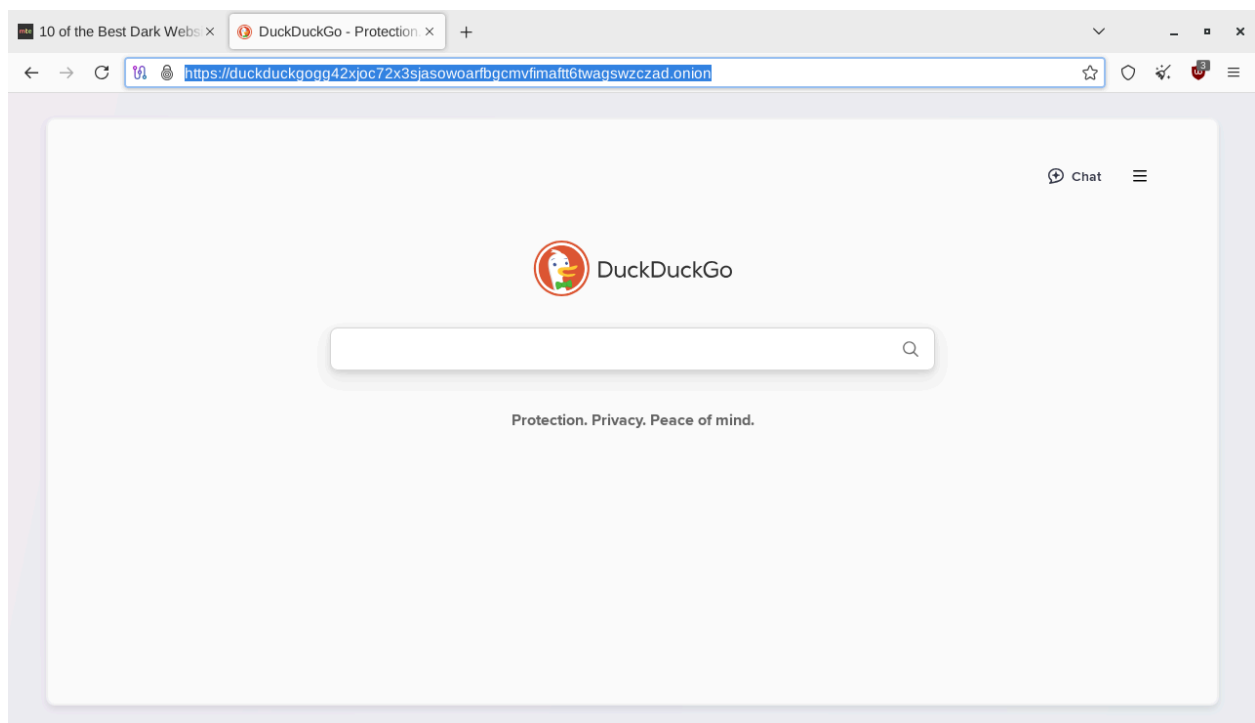


This image tells us that our browser is successfully connected to the Tor circuit browser. And also tells us our ip appears as '109.70.100.4'.

Now after we are done with checking our browser is onionized. We are going to check DuckDuckGo onion site, and the address for the onion search engine is "<https://3g2upl4pg6kufc4m.onion/>" By clicking this link we can see the onion site of DuckDuckGo. Now going to the same link from the unsafe browser presented in Tails. When we try to visit the link we will get the message saying " Hmm. We're having trouble finding that site, which means that we are not the onionized browser and to visit onion sites we have to be on the Tor circuit browser. But I also tried to visit [Google.com](https://www.google.com/) and took me there successfully, so it clearly means that .onion sites are not part of the regular internet. They are only accessible through the Tor network, which uses a specialized protocol (Onion Routing) to anonymize traffic and hide both users and servers.



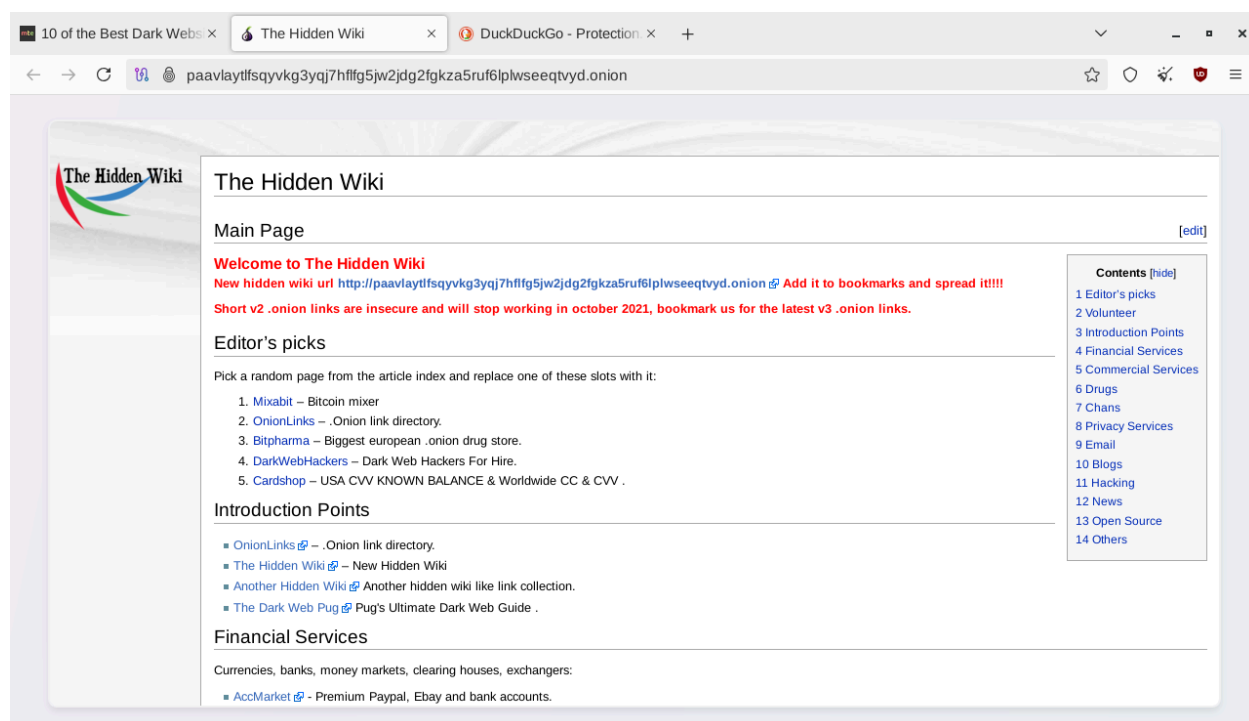
This above image shows us accessing the onion site using unsafe browser we can access the [google.com](https://www.google.co.in/) and couldn't access the onion site.



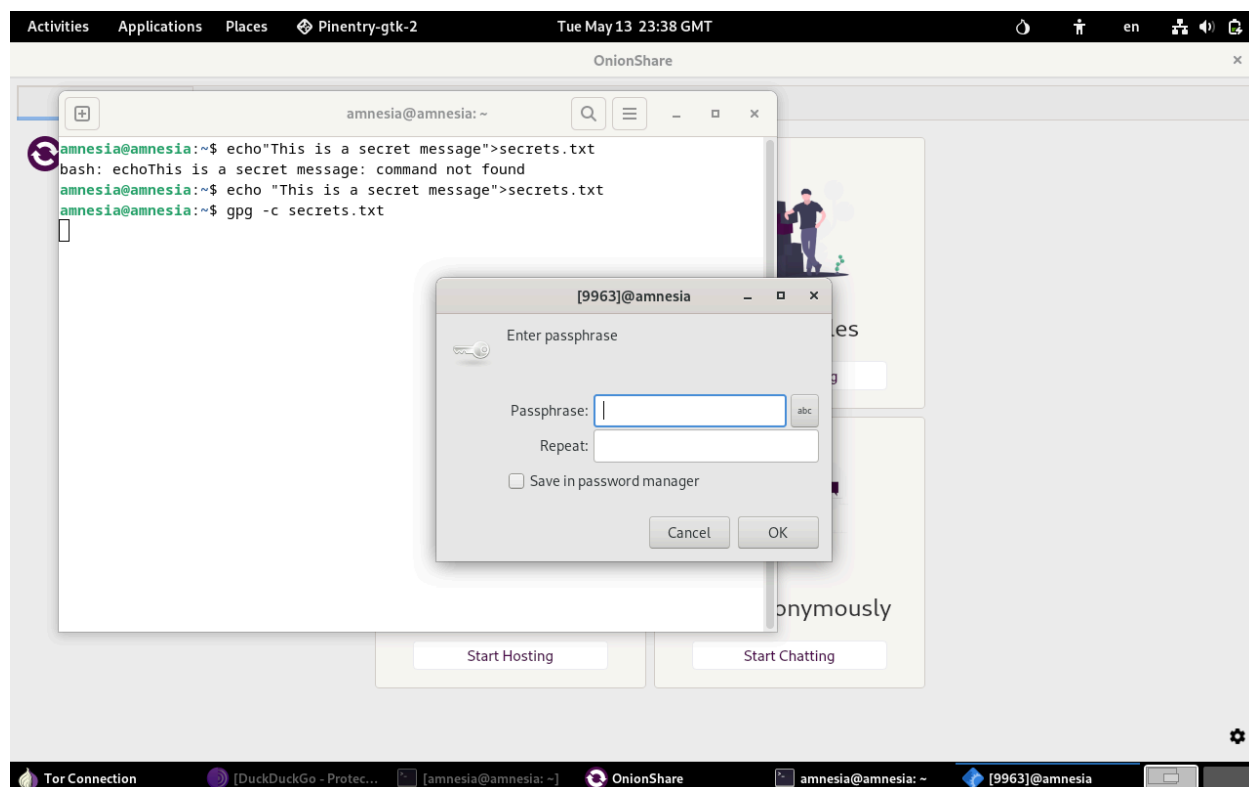
The above image shows that we can access the .onion site using the Tor network and also shows e=we have successfully landed DuckDuckGo Tor site.

And also tried to visit a few sites on Onion browser like the Hidden WikiThe Hidden Wiki is a community-curated website that provides categorized links to various .onion services available only through the Tor network. Since .onion addresses are not indexed

by traditional search engines like Google, the Hidden Wiki offers a convenient way for users to discover different Tor-based resources, including forums, marketplaces, blogs, secure messaging services, and privacy tools. It's similar to a directory or portal, listing links and short descriptions. While some entries are legitimate or educational, others may point to illegal or unethical content. Because it's editable and decentralized (there are many versions), users should exercise extreme caution when visiting sites listed there. as shown below



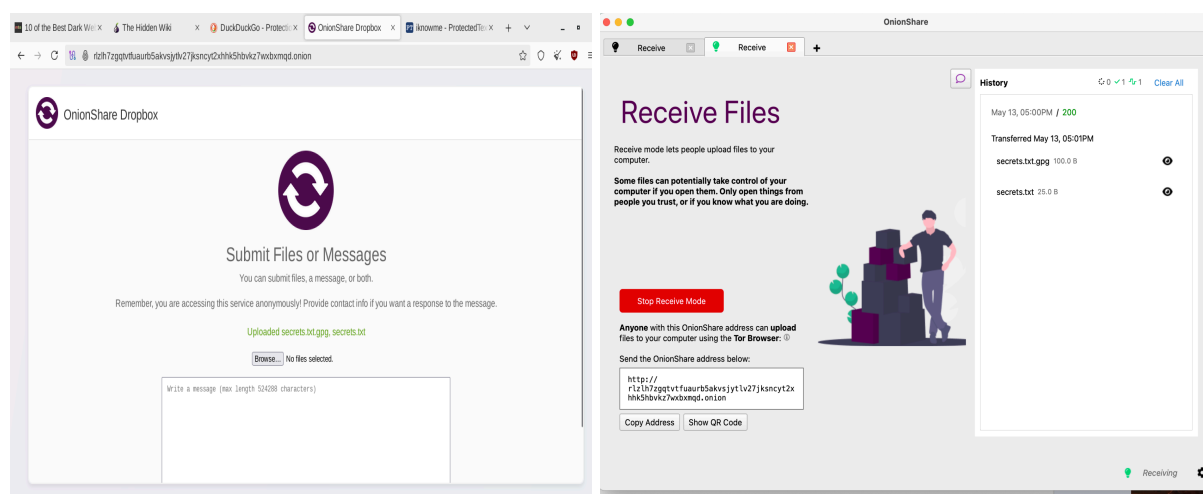
Afterwards, we are creating a text file called secrets.txt using the terminal in Tails Os. Go to terminal and type the command 'echo "This is a secret message">secrets.txt' this will create a file named secrets.txt and write the text "This is a secret message" into it. The echo command outputs the given string, and the > symbol is used to redirect that output into a file. Then after creating the secrets.txt file we are going to encrypt the contents using symmetric encryption. We are going to type the command " gpg -c secrets.txt." When you run this command, GPG will prompt you to enter a passphrase. This passphrase is used to encrypt the secrets.txt file using symmetric encryption, meaning the same passphrase is required to decrypt it later. Once the encryption is complete, a new file named secrets.txt.gpg is created, containing the encrypted data. The original secrets.txt file remains unchanged unless you manually delete it.



The above image shows us that we are running `gpg -c secrets.txt` to encrypt the file using GPG symmetric encryption. As a result, a passphrase prompt appears on the screen, asking the user to enter and confirm a passphrase.

When we successfully encrypt our `secrets.txt` with `gpg` symmetric encryption. Now we are going to share that file and also `tcpdump` file using Onionshare, it is a privacy-focused application that uses the Tor network to allow users to share files, host temporary websites, or conduct private chats without exposing their identity or location. When you use OnionShare, it generates a unique, temporary .onion URL that others can open in the Tor Browser to download your files or chat with you. The connection is direct and encrypted, with no need for a central server. Since it's based on Tor, it protects your IP address, supports censorship resistance, and ensures anonymous access. OnionShare is especially useful for journalists, whistleblowers, activists, or anyone who needs to transmit sensitive information securely. It offers modes for sending files, receiving files, anonymous chat, and static site hosting all without requiring any account, registration, or third-party service. Now we are going to download the onion share for our imac, after downloading it it will automatically connect to the Tor network when started and shows two option share or receive, we click on receive and now the OnionShare application running in Receive mode, which allows the user to securely receive files from others via the Tor network. We get the receiver link as "`http://rlzh7zgqvtfuau...ksnc yt2xhhk5hbvkz7wxbxm q d.onion`" Once we have that link we are going to send it from Tails, Now we are going to the onion browser and type in

the link it will prompt us the sending interface to select files. Now we select the secrets.txt and secrets.txt.gpg file and send it.



The first image shows us the sending interface and the second image shows us the receiver interface.

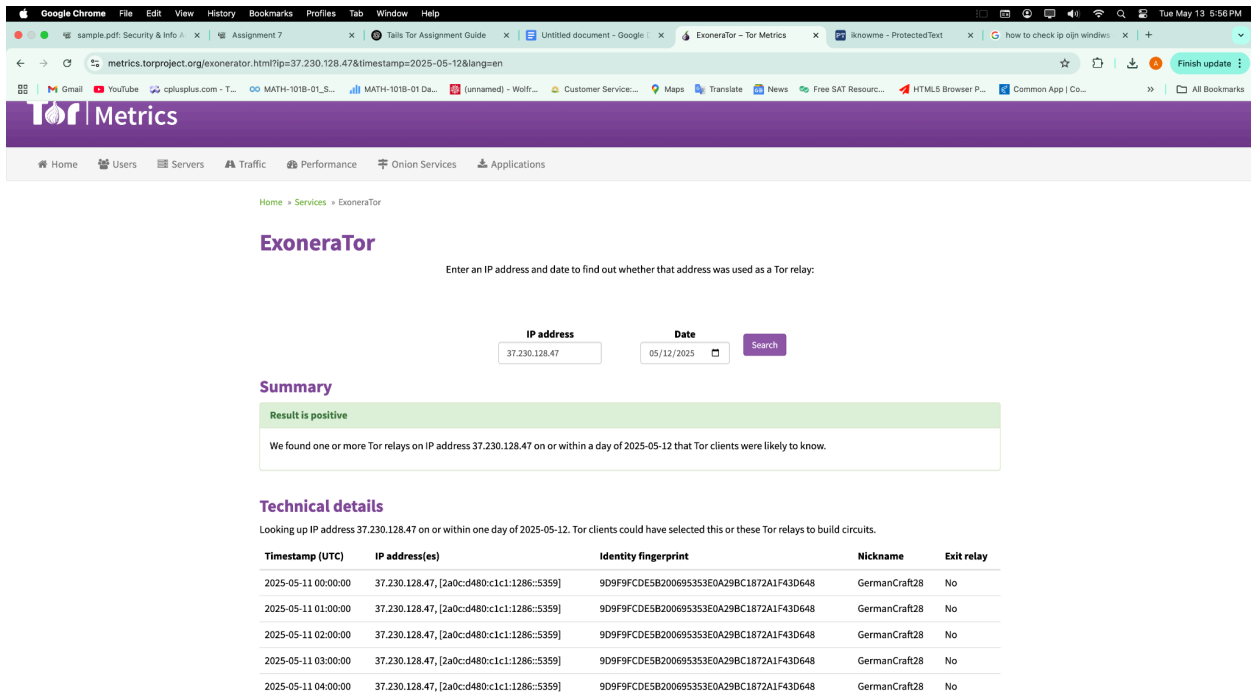
And after we send the file we have to end the tcp capture and change its permission using “`sudo chown amnesia:amnesia /home/amnesia/Desktop/packets.pcap`” `sudo chmod 777 /home/amnesia/Desktop/packets.pcap`” The commands change the ownership of the file packets.pcap so that the user amnesia and the group amnesia have control over it. This is useful when a file was created or captured by a tool run with root privileges, such as tcpdump, and the regular user cannot open or modify it. The second command, `sudo chmod 777 /home/amnesia/Desktop/packets.pcap`, sets the file permissions to 777, which means read, write, and execute permissions are granted to the owner, the group, and all other users. While this ensures maximum accessibility for the file. After we have the packets.pcap file we are going to share it using the same way using the onion share and get it in our imac for further investigation in the captured files.

```
amnesia@amnesia: ~  
12: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:dc:c4:db brd ff:ff:ff:ff:ff:ff permaddr 08:00:27:c6:89:1d  
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0  
        valid_lft 86160sec preferred_lft 86160sec  
    inet6 fe80::f252:e1b1:c0ef:ca21/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
amnesia@amnesia:~$ tcpdump -i any -s 65535 -w /home/amnesia/Desktop/packets.pcap  
tcpdump: any: You don't have permission to perform this capture on that device  
(socket: Operation not permitted)  
amnesia@amnesia:~$ sudo -i  
[sudo] password for amnesia:  
root@amnesia:~# tcpdump -i any -s 65535 -w /home/amnesia/Desktop/packets.pcap  
tcpdump: data link type LINUX_SLL2  
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 65535 bytes  
  
^C24599 packets captured  
27505 packets received by filter  
0 packets dropped by kernel  
root@amnesia:~# sudo chown amnesia:amnesia /home/amnesia/Desktop/packets.pcap  
root@amnesia:~# sudo chmod 777 /home/amnesia/Desktop/packets.pcap  
root@amnesia:~#
```

The above image shows us we are typing in the command to change the packets.pcp file owner from root.

Now after everything is sent we are going to do analysis of the captured files in wireshark of Kali linux. We have done two network captures, one in wireshark and the other in tcp dump. First let's see what was going on tcp dump capture. We see a bunch of onion nodes because our connection is not bridged. We and our isp can see we are connecting Tor but they cannot see what we are doing on it. We select one ip "37.230.128.47" and check if it is Tor node or not, Now we go to a website called Tor Project's ExoneraTor tool, which is used to determine whether a specific IP address was part of the Tor network on a given date. In this case, the IP address 37.230.128.47 was checked for activity on May 12, 2025. The result was positive, meaning this IP was indeed functioning as a Tor relay specifically, a non-exit relay on or around that date. This indicates that the IP was involved in forwarding encrypted Tor traffic within the network but was not the final node that connects to the open internet. The technical details list the relay's identity fingerprint and nickname (GermanCraft28), along with multiple timestamps showing its active presence. This information is useful in network

analysis, especially when investigating .pcap files from tools like Wireshark for tcpdump. If this IP appears in captured packets, it confirms that the traffic was routed through the Tor network, which could be associated with activities using tools like the Tor Browser, Tails OS, or OnionShare.



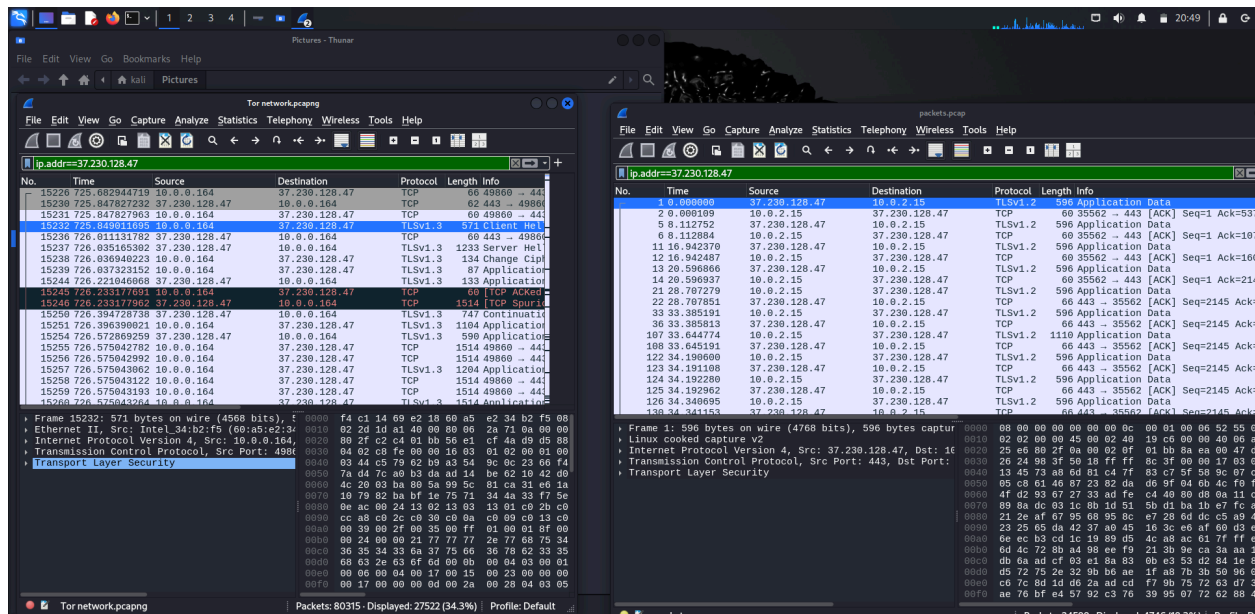
Here is the image showing that the ip-address “37.230.128.47” is listed on Tor network.

When analyzing Tor-related traffic, especially using tools like Wireshark on a host operating system and tcpdump inside a Tails virtual machine, it’s common to see differences in source or destination IP addresses. This happens because the network interface seen by the host Windows or Kali and the VM Tails OS can behave differently based on how the VM is configured bridged, NAT, or host-only mode. In many cases, the Tails VM running OnionShare or Tor Browser routes all traffic through the Tor network, so any outgoing connections are encrypted and relayed through multiple Tor nodes. On the Tails side, when you run tcpdump, you might see only the immediate relay IP (like the guard node) that Tor is connecting to—this is typically the entry point into the Tor network.

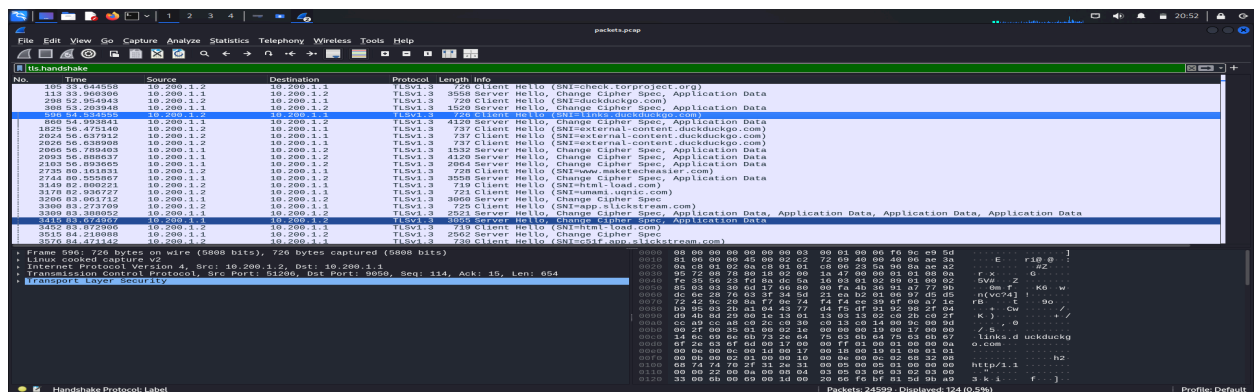
However, on the Wireshark capture running on Kali, we will be observing the external communication of the VM as seen by the host. If the VM is using NAT networking, the traffic is masqueraded meaning the source IP seen from outside (like in Wireshark) is actually that of the host system, not the internal IP of the VM. So while tcpdump inside Tails shows the correct internal VM IPs and Tor relays, Wireshark on the host sees

traffic appearing to come from the host machine itself, going to what appears to be a random IP but is actually a Tor relay, confirmed using ExoneraTor. This explains why we see the Windows IP in Wireshark captures, even though Tails is the system making the Tor connection. This behavior is not a leak Tor is still working securely but it reflects how packet capture differs depending on where and how you observe the traffic. The key takeaway is that your host sees NAT traffic, while your VM sees the raw, encrypted Tor handshake with relays.

And observing traffic we also saw that common internet protocols like TCP and TLS are used in both regular internet traffic and Onion Routing (Tor), how they are used and behave in the Tor network is significantly different due to Tor's layered encryption and relay-based architecture.



In the above image there are two wireshark tabs open, in the first one we can see that it was captured in Kali Linux in background, and the right side is tcpdump inside Tails but both are visiting to the same server but the source ip addresses are different in both.



In this image we can see the TLS handshake of the Tor circuit.

Conclusion

In this assignment, we used the Tails operating system and the Tor network to explore how anonymous communication and privacy tools work in real world settings. Through packet capture, file encryption, and anonymous file sharing, we gained insight into how Tor routes traffic to protect users' identity and data. We started by setting up Tails in VirtualBox, enabling Live mode and allocating sufficient RAM. After booting, we connected directly to the Tor network and began capturing network traffic using tcpdump. We confirmed Tor connectivity via check.torproject.org and accessed .onion sites like DuckDuckGo and the Hidden Wiki, demonstrating how Tor enables access to hidden services. Next, we created and symmetrically encrypted a text file using `gpg -c`, then securely shared it along with the packet capture file via OnionShare. We used Tails to upload and our iMac to receive the files. File permissions were adjusted for access and the .pcap file was opened in Wireshark. Analysis confirmed our traffic passed through real Tor nodes, verified using ExoneraTor. By using Tails in a virtual environment and capturing network traffic with tcpdump, I was able to observe the effectiveness of Tor's encryption and routing mechanisms, which ensure that neither the origin nor destination of the data can be easily traced. Tor improves security by implementing onion routing, where data is encrypted in multiple layers and sent through at least three randomly chosen relays entry, middle, and exit making it extremely difficult for any observer to trace the path or content of the traffic. Beyond Tor's network-level protection, Tails offers several built-in tools that further strengthen security. These include automatic routing of all connections through Tor, a secure file-sharing tool called OnionShare, GnuPG for file and email encryption, an "Unsafe Browser" that is isolated to prevent leaks, and automatic memory erasure upon shutdown to ensure no trace of user activity remains. Additionally, optional encrypted persistent storage can be configured for saving important files securely. Using Tor inside Tails is far more secure than using Tor alone on a traditional operating system, as typical systems may leave behind logs, cache files, or other traces that can compromise anonymity. In contrast, Tails runs entirely on memory and leaves no footprint unless explicitly told to, combining system-level and network-level privacy protections. Together, Tor and Tails offer a powerful solution for anyone needing to communicate or browse the internet privately, particularly in environments where surveillance, censorship, or monitoring are concerns.